Tentamen TEN1 HI1029 2024-03-13

Sista ordinarie insläpp: 8.00 – dörren stängs.
Tentavakterna delar ut konton. Logga in i tentasystemet så fort som möjligt.
När alla loggat in (förhoppningsvis 8.15) börjar tentamen och pågår i 4h 45min (tex 8.15-13.00).
8.30 Insläpp för försenade.

Extra skrivtid 50%: 7h 10min (tex 8.15-15.25) *(endast 1h extra skrivtid för A eller B)*
Skrivvakter: Anteckna start- och inlämningstid. Jour: Nicklas Brandefelt 087909738

Hjälpmedel: papper (tomma), penna, mus

Logga in med tentamenskontot ni får av skrivvakten. Logga inte ut förrän ni är klara med hela tentan.

Skapa ett nytt projekt i IntelliJ: **Project Name:** TEN1, **Project Location:** H

Skapa ett package per uppgift och kalla dessa uppgift1o2, uppgift3, uppgift4, uppgift 5, uppgift6. Kopiera hjälpfiler från P/HI1029/ till rätt package.

Svaren på teorifrågorna (fråga ett och två) skall besvaras i en textfil döpt till teori.txt som ni lägger i package uppgift1o2.

Även om ni inte löser hela uppgiften så kan ni få delpoäng men ni måste se till att koden kompilerar och kan köras. Både output och koden bedöms och det är viktigt att denna är välskriven och uppfyller uppgiftens instruktioner. Om ett program är ineffektivt kan detta ge poängavdrag. Det är meningen att man skall använda de tekniker vi lärt oss där detta är fördelaktigt. Välj bra namn för variabler, metoder och klasser. Se till att programmet har en bra struktur. Ni behöver på denna tentamen inte kommentera.

Hjälpfiler hittar ni i P/HI1029/ och Java API i P/Java/api/index.html.

Del 1 (15p): För godkänt krävs minst 1 poäng på teorifrågorna samt 9 poäng totalt. Om man får minst en poäng på teorifrågorna får man betyg enligt: 9p – E, 10p – D, 12p – C. Om man inte får minst en poäng på teorifrågorna men minst 8,5 poäng totalt får man Fx.

Del2: Bedöms bara om man får betyg C på del 1 och bedöms då med A, B eller C. Observera att om du har extra skrivtid får du bara använda 1 extra timme för att kunna få betyg A eller B.

Examinator och rättande lärare: Nicklas Brandefelt

Del 1

1. Vad är för och nackdelar med en rekursiv lösning jämfört med en iterativ? (1p)
   Svaret skall lämnas i en textfil, teori.txt i package uppgift1o2.

2. Ange och motivera ordo för insättning och borttagning i en heap. (1p) *Svaret skall lämnas i en textfil, teori.txt i package uppgift1o2.*

3. ***Ni som har fått godkänt på krysstalen i år hoppar över denna uppgift. Ange B på försättsbladet i rutan för uppgift 3.***
   Implementera en stack för data av typen String med hjälp av en enkellänkad lista som inre datastruktur. Du får inte använda någon datastruktur från Java's API (men får naturligtvis skapa egna klasser). Den ska förutom konstruktorn ha fyra publika metoder: pop, peek, push och isEmpty. Dessa ska alla uppföra sig så som man kan förvänta sig av en stack och de ska alla ha O(1). Om man anropar pop eller peek på en tom stack ska dessa returnera null. Du hittar en main-klass som ska fungera med din stack i P/HI1029/uppgift3. (3p)

4. I denna uppgift ska vi ta oss från ett givet heltal till talet ett i så få steg som möjligt om det är möjligt. I varje steg får vi göra någon av följande operationer (om de är tillåtna):

   - Om talet är delbart med 7 får vi subtrahera 20 (a)
   - Om talet är delbart med 5 får vi subtrahera 16 (b)
   - Om talet är delbart med 2 får vi halvera det (c)
   - Vi kan alltid välja att subtrahera 7 (d)

   Om vi börjar med talet 70 så kan man nå ett i 4 steg genom att välja c, d, d och sist a (70/2=35, 35-7=28, 28-7=21, 21-20=1). Om vi istället börjar med 623 så krävs 9 steg: dcccdcdda. I denna uppgift ska du skriva metoder som returnerar antal steg men inte presenterar vilka steg som behöver göras. Enkla skal hittar du i P/HI1029/uppgift4.

   a) Skriv en metod som tar ett heltal som in-parameter och returnerar antal steg som krävs för att nå 1 enligt ovan. Om lösning saknas ska metoden returnera -1. Använd en rekursiv djupet först lösning. (2p)

   b) Effektivisera din lösning med hjälp av dynamisk programmering. Gör denna i en separat metod. (2p)

   c) Skriv nu ännu en metod som löser samma problem men den här gången ska metoden lösa problemet med bredden först. Även denna ska returnera -1 om lösning saknas. (2p)

5. I den här uppgiften utgår vi ifrån vårt binära sökträd från föreläsningen (förenklat). Utgå ifrån filerna i P/HI1029/uppgift5. Kom ihåg att metoderna ska kunna hantera även ett tomt träd. Försök skriva dem så effektiva som möjligt. Du ska inte lägga till några medlemsvariabler men får använda privata hjälpmetoder.

   a) Skriv metoden height som returnerar trädets höjd. (1p)

   b) Skriv metoden height som tar ett data av typen E och returnerar höjden hos det subträd vars rotnod innehåller datat. Om datat saknas i trädet returnerar den noll. (1p)

   c) Skriv metoden siblings som tar två data och returnerar true om båda finns i trädet och har samma förälder och annars false. Om båda data är lika ska den returnera false. (2p)

Del 2

6. Skriv en metod som returnerar det minsta antalet tidssteg som krävs för att gå från start ('S') till mål ('M') i en labyrint. Att flytta från en ruta till en angränsande ruta tar ett tidssteg (det är ej tillåtet att flytta diagonalt). På vissa rutor finns fällor som tar ett visst antal tidssteg att komma loss från. Dessa anges med en siffra som betecknar hur länge man måste vänta innan man kan gå vidare. En 2:a betyder att man måste vänta två tidssteg innan man kan gå vidare. '*' markerar en vägg och får inte beträdas. Labyrinten är alltid omgiven av en kant med '*' så du behöver inte hantera att du kan ramla över kanten. Utgå ifrån filen i katalogen P/HI1029/uppgift6. I exemplet nedan är en väg med minsta antalet tidssteg markerad. Att gå denna tar 25 tidssteg. Försök skriva din kod så effektiv som möjligt. En alltför ineffektiv lösning kommer att bedömas med betyget C.

*	*	*	*	*	*	*	*	*
*				*			M	*
*		*		*	2			*
*		*		6	7	1	2	*
*		*			*	*		*
*		*			5			*
*	S	8	7			2		*
*	*	*	*	*	*	*	*	*